

Transformer Models: Encoder-Decoder, Encoder-Only, and Decoder-Only

Three Main Transformer Settings

Model type	Information flow	Main use	Typical training signal
Encoder-decoder	input is encoded, output is decoded with cross-attention	sequence-to-sequence tasks such as translation and summarization	next-token loss on the target sequence
Encoder-only	full bidirectional self-attention over the input	understanding and representation learning	masked-token prediction or supervised task loss
Decoder-only	causal self-attention, left-to-right generation	language modeling and text generation	autoregressive next-token prediction

Encoder-Decoder vs Encoder-Only vs Decoder-Only

Encoder-decoder

$$x_{1:n} \xrightarrow{\text{encoder}} H_{1:n}, \quad y_t \sim p(y_t \mid y_{<t}, H_{1:n})$$

Used when the input and output are **different sequences**.

Encoder-only

$$x_{1:n} \xrightarrow{\text{bidirectional encoder}} h_{1:n}$$

Used when the goal is to **understand the input** and produce contextual features.

Decoder-only

$$p(x_{1:n}) = \prod_{t=1}^n p(x_t \mid x_{<t})$$

Used when the goal is to **generate tokens one by one**.

Encoder-Only: What It Computes

For input tokens $x_1, \dots, x_n$, the encoder produces contextual vectors

$$H = [h_1, \dots, h_n] \in \mathbb{R}^{n \times d_{model}}$$

where each h_t depends on the **whole input sequence**.

Key property: token t can use both left and right context:

$$h_t = f(x_1, \dots, x_{t-1}, x_t, x_{t+1}, \dots, x_n)$$

This is why encoder-only models are strong for:

- classification,
- token tagging,
- retrieval and embedding models,
- masked language modeling.

Encoder-Only Objective Function

The standard pretraining objective is **masked language modeling (MLM)**.

Choose a set of masked positions $M \subseteq \{1, \dots, n\}$. The model sees the corrupted sequence $\tilde{x}_{1:n}$ and predicts the original tokens only at masked locations.

$$\mathcal{L}_{\text{MLM}} = - \sum_{t \in M} \log p_{\theta}(x_t \mid \tilde{x}_{1:n})$$

Interpretation:

- the encoder is bidirectional,
- prediction at position t can use context from **both sides**,
- the model is not trained to generate the whole sentence left-to-right.

For downstream classification, one often adds a task loss such as

$$\mathcal{L}_{\text{cls}} = - \log p_{\theta}(y \mid h_{[\text{CLS}]})$$

Decoder-Only: What It Computes

A decoder-only model predicts the next token using **only previous tokens**.

$$p(x_{1:n}) = \prod_{t=1}^n p_{\theta}(x_t \mid x_{<t})$$

At position t , the representation is causal:

$$h_t = g(x_1, \dots, x_t)$$

It cannot use $x_{t+1}, x_{t+2}, \dots$ because of the causal mask.

This makes decoder-only models natural for:

- text generation,
- dialogue,
- code completion,
- general autoregressive language modeling.

Decoder-Only Objective Function

The standard training objective is the **autoregressive next-token loss**:

$$\mathcal{L}_{\text{AR}} = - \sum_{t=1}^n \log p_{\theta}(x_t \mid x_{<t})$$

Equivalently, training maximizes the log-likelihood of the full sequence:

$$\log p_{\theta}(x_{1:n}) = \sum_{t=1}^n \log p_{\theta}(x_t \mid x_{<t})$$

Difference from encoder-only:

- encoder-only predicts selected masked tokens using **both-side context**,
- decoder-only predicts the next token using **left context only**.

Goals

In this lecture we want to understand **both** major single-stack Transformer families:

- **encoder-only**: best thought of as a contextual feature extractor,
- **decoder-only**: best thought of as a conditional next-token generator.

We will still spend slightly more time on the encoder block because it is a clean starting point, but we will also cover:

- decoder-only architecture,
- causal masking,
- autoregressive generation,
- how the two objective functions shape the representations.

Encoder-Only Transformer: High-Level Architecture

For a sequence of length n :

tokens $\rightarrow$ embeddings $\rightarrow$ add positions $\rightarrow$ $\underbrace{\text{Encoder Block} \rightarrow \dots \rightarrow \text{Encoder Block}}_{L \text{ times}} \rightarrow$ contextual outputs

If $X^{(0)} \in \mathbb{R}^{n \times d_{\text{model}}}$ is the input matrix, then each block keeps the same outer shape:

$$X^{(\ell)} \in \mathbb{R}^{n \times d_{\text{model}}} \quad \text{for } \ell = 0, 1, \dots, L$$

A standard encoder block contains:

- 1 multi-head self-attention,
- 2 residual connection + LayerNorm,
- 3 position-wise feed-forward network,
- 4 residual connection + LayerNorm.

Decoder-Only Transformer: High-Level Architecture

The decoder-only stack looks similar, but the self-attention is **causal**.

tokens $\rightarrow$ embeddings $\rightarrow$ add positions $\rightarrow$ $\underbrace{\text{Decoder Block} \rightarrow \dots \rightarrow \text{Decoder Block}}_{L \text{ times}} \rightarrow$ logits over vocabulary

If $X^{(0)} \in \mathbb{R}^{n \times d_{\text{model}}}$, then every block again preserves shape:

$$X^{(\ell)} \in \mathbb{R}^{n \times d_{\text{model}}}$$

A standard decoder-only block contains:

- 1 masked multi-head self-attention,
- 2 residual connection + LayerNorm,
- 3 position-wise feed-forward network,
- 4 residual connection + LayerNorm.

The main architectural difference is the **causal mask**, not a completely different block design.

Causal Masking in Decoder-Only Models

In an encoder, token t may attend to all positions $1, \dots, n$.

In a decoder-only model, token t may attend only to positions $1, \dots, t$.

$$M_{tj} = \begin{cases} 0, & j \leq t \\ -\infty, & j > t \end{cases}$$

Then the attention scores use

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}} + M\right)$$

so future positions get probability 0.

Why this matters:

- training matches generation,
- the model cannot “cheat” by reading future tokens,
- each hidden state is a summary of the prefix only.

Encoder-Only vs Decoder-Only at a Glance

Question	Encoder-only	Decoder-only
What context can token t use?	left and right context	left context only
Main pretraining loss	MLM	autoregressive next-token loss
Main output style	contextual representation	generated token distribution
Typical tasks	classification, tagging, retrieval	generation, dialogue, completion
Attention mask	none for standard self-attention	causal triangular mask

Why Do We Need Positional Encoding?

Self-attention alone is **permutation-equivariant**: if we reorder the tokens, the same reordering appears in the outputs.

Without positional information:

- “dog bites man”
- “man bites dog”

contain the same words but have different meanings.

So we inject position into each token representation:

$$X_{pos} = E_{pos} + PE_{pos}$$

where

- $E_{pos} \in \mathbb{R}^{d_{model}}$ is the token embedding,
- $PE_{pos} \in \mathbb{R}^{d_{model}}$ is the positional vector.

Then the model sees both **what** the token is and **where** it is.

Sinusoidal Positional Encoding

For position pos and model dimension d_{model} :

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

What is the i ?

- The embedding coordinates are grouped into pairs:

$$(0, 1), (2, 3), (4, 5), \dots$$

- $i = 0, 1, \dots, \frac{d_{model}}{2} - 1$ indexes the **pair number**.
- Pair $(2i, 2i + 1)$ uses one sine-cosine frequency.

So i is not the token position. It is the **frequency-channel index** inside the embedding vector.

Positional Encoding with More Math

Define the angle for pair i at position pos as

$$\theta_{pos,i} = \frac{pos}{10000^{2i/d_{model}}}$$

Then the two coordinates in that pair are

$$PE_i(pos) = \begin{bmatrix} \sin(\theta_{pos,i}) \\ \cos(\theta_{pos,i}) \end{bmatrix}$$

Why different i values matter:

- small $i \Rightarrow$ small denominator $\Rightarrow$ angle changes quickly with position,
- large $i \Rightarrow$ large denominator $\Rightarrow$ angle changes slowly with position.

So lower-index pairs capture **fine local order**, while higher-index pairs capture **coarser global position**.

Why Sine and Cosine Are Useful

If we shift position by k , then

$$\theta_{pos+k,i} = \theta_{pos,i} + \frac{k}{10000^{2i/d_{model}}}$$

Using angle-addition identities,

$$\begin{bmatrix} \sin(\theta_{pos+k,i}) \\ \cos(\theta_{pos+k,i}) \end{bmatrix} = \begin{bmatrix} \cos \Delta_i & \sin \Delta_i \\ -\sin \Delta_i & \cos \Delta_i \end{bmatrix} \begin{bmatrix} \sin(\theta_{pos,i}) \\ \cos(\theta_{pos,i}) \end{bmatrix}$$

where

$$\Delta_i = \frac{k}{10000^{2i/d_{model}}}$$

Interpretation: moving by k positions corresponds to a **rotation** in each 2D sine-cosine plane.

Multi-Head Self-Attention: Main Idea

We will keep the attention-score details light.

Each head makes its own projections:

$$Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V$$

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

The heads are concatenated and mixed:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

If d_{model} is split evenly across h heads, then

$$d_{\text{head}} = \frac{d_{\text{model}}}{h}$$

Each head works in a smaller learned subspace.

Why Use Multiple Heads?

- One head gives one interaction pattern.
- Many heads let the model build **different interaction patterns in parallel**.

Different heads may specialize in different relations, for example:

- short-range dependencies,
- long-range dependencies,
- syntax-like relations,
- entity references,
- phrase boundaries.

Conceptually:

- each head sees a different projection of the same tokens,
- each head computes its own token-to-token mixing,
- concatenation combines these views into one richer representation.

Feed-Forward Layer, Residual Path, and LayerNorm

After attention, each token is passed through the same position-wise feed-forward network:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

where σ is often ReLU or GELU.

Residual connections

- preserve a clean information path,
- help optimization,
- make deep stacks easier to train.

LayerNorm

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

where μ and σ^2 are computed across the feature dimension of one token, and γ, β are learned parameters.

LayerNorm Placement: Before or After Addition?

There are two common designs.

Post-LN

$$Y = \text{LN}(X + \text{MHA}(X)), \quad Z = \text{LN}(Y + \text{FFN}(Y))$$

Pre-LN

$$Y = X + \text{MHA}(\text{LN}(X)), \quad Z = Y + \text{FFN}(\text{LN}(Y))$$

Interpretation:

- Post-LN normalizes **after** the residual addition.
- Pre-LN normalizes **before** the sublayer, then adds the residual.
- Pre-LN is often easier to optimize in deep models because the residual path stays more direct.

How Should We Initialize the Model?

A practical initialization recipe:

- **Linear layers** (W^Q, W^K, W^V, W^O and FFN weights): Xavier/Glorot is a strong default.
- **Biases**: initialize to zero.
- **Token embeddings**: small random values, often from a normal distribution.
- **LayerNorm**: initialize

$$\gamma = 1, \quad \beta = 0$$

- **Fixed sinusoidal positional encoding**: no learned parameters, so no initialization is needed.
- **Learned positional embeddings**: initialize like token embeddings.

Why this works: keep activations well-scaled, keep the residual stream stable, and avoid large initial magnitudes that make optimization harder.

How Decoder-Only Generation Works at Inference

Training uses the full sequence in parallel with a causal mask, but inference generates **one token at a time**.

Starting from a prompt $x_{1:t}$:

$$p_{\theta}(x_{t+1} \mid x_{1:t})$$

Choose the next token, append it to the prefix, and repeat:

$$x_{1:t} \rightarrow x_{1:t+1} \rightarrow x_{1:t+2} \rightarrow \dots$$

Important point:

- the same autoregressive factorization is used in training and generation,
- hidden states summarize prefixes,
- generation is sequential because the next token is unknown until the previous prediction is made.

Summary

- **Encoder-only** models use bidirectional context and are commonly pretrained with

$$\mathcal{L}_{\text{MLM}} = - \sum_{t \in M} \log p_{\theta}(x_t \mid \tilde{x}_{1:n})$$

- **Decoder-only** models use causal context and are trained with

$$\mathcal{L}_{\text{AR}} = - \sum_{t=1}^n \log p_{\theta}(x_t \mid x_{<t})$$

- In sinusoidal positional encoding, i indexes the **dimension pair / frequency band**, not the token position.
- Multi-head attention means parallel attention in different learned subspaces.
- Encoder-only and decoder-only blocks look similar, but the **attention mask and objective function** change what the model learns.